TP 2 — Sécurité dans un pipeline CI/CD · fiche de soutenance

VulnBank · GL4 Groupe 1 · `github.com/HanineKhemir/devsecops-tp2-cicd`

L'idée en une phrase

Au TP1 nous lancions les scanners **à la main** ; au TP2 c'est **le dépôt qui refuse** le code vulnérable — la sécurité devient un contrôle d'accès, pas une bonne intention.

Le pipeline : 4 portes en parallèle + 1 verdict

#	Porte	Outil	Règle de blocage
1	Build & tests	pytest	un test rouge
2	**SAST**	Bandit	≥ 1 constat **HIGH**
3	**SCA**	pip-audit / OSV	≥ 1 **CVE connue**
4	**DAST**	OWASP ZAP (scan **actif**)	≥ 1 alerte **High** (`riskcode 3`)

Le job `security-gate` agrège les 4 → déclaré **check requis** sur `main` → un pipeline rouge **désactive le bouton de fusion**.

Trois choix à défendre

1. **Les règles sont des scripts** (`scripts/gate_*.py`), pas le code de sortie des scanners. Un scanner échoue sur *tout*, même informatif : le pipeline serait rouge en permanence, donc ignoré. Nos scripts : règle écrite, exceptions justifiées et datées, et **rapport manquant = porte rouge** (un scanner cassé ≠ un scanner qui n'a rien trouvé).
2. **Jobs indépendants**, pas en chaîne. Sinon des tests rouges empêcheraient les scanners de tourner et une PR vulnérable ne montrerait qu'une faille à la fois.
3. **Scan actif, pas baseline.** Le scan passif ne produit presque jamais d'alerte High : il ne verrait ni l'injection SQL ni le XSS. Et on **attend le HEALTHCHECK** — scanner une app pas démarrée donne un rapport vide, donc une porte **verte à tort**.

L'environnement éphémère (porte DAST)

`docker build` → réseau privé `zapnet` → conteneur **non exposé** → attente du HEALTHCHECK → ZAP `zap-full-scan.py` → verdict → `docker rm -f` (`if: always()`). Né et détruit dans le run. Rejouable en local : `docker-compose.ci.yml` .

La démonstration : 2 pull requests

	PR #1 `feat(search)`	PR #2 `fix(search)`
Tests	❌ 4 échecs	✅ 17 passent
SAST	❌ 2 HIGH (B605, B201)	✅ 0
SCA	❌ 10 CVE (`urllib3 1.26.5`)	✅ 0
DAST	❌ **5 alertes High**	✅ 0 High
GitHub	**BLOCKED**	**CLEAN**

La même fonctionnalité `/search` : refusée écrite sans précaution, acceptée écrite correctement.

Les deux arguments qui portent

1. Le DAST a vu ce que le SAST ne voyait pas. Bandit n'a classé la concaténation SQL qu'en **MEDIUM** (non bloquant) et n'a **rien dit du XSS**. ZAP, lui, a envoyé `zj{{2756*1993}}zj` et **a reçu le résultat du calcul** : `render_template_string()` ne donnait pas un simple XSS mais une **injection de template côté serveur (SSTI, CWE-1336)** — chemin classique vers l'exécution de code à distance. *Sans la porte DAST, trois failles passaient.*

2. Le pipeline a trouvé une faille que nous n'avions pas introduite. Au premier run, la porte SCA a bloqué sur `certifi 2024.8.30` : version saine au TP1, mais **une CVE a été publiée depuis** (GHSA-6q48-6c7w-9gw2, autorité de certification compromise). Le code n'a pas bougé, le risque si. Un scan manuel une fois par semestre ne l'aurait jamais vu.

Limites assumées

ZAP signale encore **4 alertes MEDIUM** sur `main` (pas de CSP, pas de jetons anti-CSRF, clickjacking) : nous ne bloquons pas dessus. **Un pipeline vert définit un plancher, pas une application sûre.** La politique SCA stricte peut bloquer le dépôt pour une CVE qui ne nous concerne pas — d'où le mécanisme d'exceptions versionné plutôt qu'un assouplissement de la règle.

Vocabulaire

SAST lit le code · **SCA** lit les dépendances · **DAST** attaque l'application qui tourne · **Porte (quality gate)** décide si le pipeline continue · **Faux négatif** = la faille réelle qu'on n'a pas signalée, le cas le plus dangereux.